Tree Algorithms

> → **Key Idea**: Leverage tree properties like hierarchy and recursion for efficient solutions

> • **When to use**: Tree traversal, BST operations, path problems, tree construction

> ⊕ **Complexity**: Most operations O(h) where h is height, O(n) for traversal

> ⚠ **Common Pitfall**: Not handling null nodes or unbalanced trees leading to O(n) height

Overview

Tree algorithms operate on hierarchical data structures. Key patterns include traversals, BST operations, path finding, and tree properties like height and diameter.

Visual Example

Tree Traversal Orders

```
Binary Tree:
        1
       / \
      2   3
     / \   \
    4   5   6

Inorder (Left-Root-Right):    4 → 2 → 5 → 1 → 3 → 6
Preorder (Root-Left-Right):   1 → 2 → 4 → 5 → 3 → 6
Postorder (Left-Right-Root):  4 → 5 → 2 → 6 → 3 → 1
Level Order (BFS):            1 → 2 → 3 → 4 → 5 → 6
```

Binary Search Tree (BST) Property

```
Valid BST:                    Invalid BST:
        8                             8
       / \                           / \
      3   10                        3   10
     / \    \                      / \    \
    1   6    14                   1   9    14
       / \   /                       / \   /
      4   7 13                      4   7 13
```

```
Left subtree < Root < Right subtree
Node 9 violates: 9 > 8 but in left subtree
```

Tree Height and Diameter

```
Tree:
        1
       / \
      2   3
     / \
    4   5
   /
  6

Height = 3 (longest path from root to leaf: 1→2→4→6)

Diameter = 4 (longest path between any two nodes)
Path: 6 → 4 → 2 → 1 → 3  OR  6 → 4 → 2 → 5

Diameter may or may not pass through root!
```

Lowest Common Ancestor (LCA)

```
Tree:
        3
       / \
      5   1
     / \ / \
    6  2 0  8
      / \
     7   4

LCA(6, 4) = 5    (5 is ancestor of both 6 and 4)
LCA(6, 8) = 3    (3 is root, ancestor of all)
LCA(5, 4) = 5    (5 is ancestor of 4, and is itself)

Algorithm: Find paths from root, last common node in both paths
```

Tree Serialization/Deserialization

```
Serialize (Preorder with nulls):
        1
       / \
      2   3
         / \
        4   5

Serialized: "1,2,null,null,3,4,null,null,5,null,null"
```

```
Deserialize: Read values, use recursion to rebuild tree
```

Core Implementation

Tree Traversals

```java
// Inorder (recursive)
public void inorder(TreeNode root, List<Integer> result) {
    if (root == null) return;
    inorder(root.left, result);
    result.add(root.val);
    inorder(root.right, result);
}

// Preorder (iterative with stack)
public List<Integer> preorder(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null) return result;

    Stack<TreeNode> stack = new Stack<>();
    stack.push(root);

    while (!stack.isEmpty()) {
        TreeNode node = stack.pop();
        result.add(node.val);
        if (node.right != null) stack.push(node.right);
        if (node.left != null) stack.push(node.left);
    }
    return result;
}

// Level Order (BFS)
public List<List<Integer>> levelOrder(TreeNode root) {
    List<List<Integer>> result = new ArrayList<>();
    if (root == null) return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);

    while (!queue.isEmpty()) {
        int size = queue.size();
        List<Integer> level = new ArrayList<>();
        for (int i = 0; i < size; i++) {
            TreeNode node = queue.poll();
            level.add(node.val);
            if (node.left != null) queue.offer(node.left);
            if (node.right != null) queue.offer(node.right);
        }
        result.add(level);
    }
```

```
        return result;
    }
```

Tree Height and Diameter

```
public int maxDepth(TreeNode root) {
    if (root == null) return 0;
    return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
}

private int diameter = 0;
public int diameterOfBinaryTree(TreeNode root) {
    height(root);
    return diameter;
}

private int height(TreeNode node) {
    if (node == null) return 0;
    int left = height(node.left);
    int right = height(node.right);
    diameter = Math.max(diameter, left + right); // Update diameter
    return 1 + Math.max(left, right);
}
```

Time & Space Complexity

Operation	Time Complexity	Space Complexity
Tree Traversal	O(n)	O(h)
BST Search	O(log n) avg, O(n) worst	O(h)
Tree Construction	O(n)	O(n)
Path Problems	O(n)	O(h)
LCA	O(n) or O(log n)	O(h)

Where n = number of nodes, h = height of tree.

Common Problem Patterns

1. Validate Binary Search Tree

Problem: Check if a binary tree is a valid BST.

```
public boolean isValidBST(TreeNode root) {
    return validate(root, null, null);
}

private boolean validate(TreeNode node, Integer min, Integer max) {
    if (node == null) return true;
    if ((min != null && node.val <= min) || (max != null && node.val >= max)) return
    return validate(node.left, min, node.val) && validate(node.right, node.val, max)
}
```

2. Lowest Common Ancestor

Problem: Find LCA of two nodes in a binary tree.

```
public TreeNode lowestCommonAncestor(TreeNode root, TreeNode p, TreeNode q) {
    if (root == null || root == p || root == q) return root;
    TreeNode left = lowestCommonAncestor(root.left, p, q);
    TreeNode right = lowestCommonAncestor(root.right, p, q);
    if (left != null && right != null) return root;
    return left != null ? left : right;
}
```

3. Binary Tree Maximum Path Sum

Problem: Find maximum path sum (path can start/end at any node).

```
private int maxSum = Integer.MIN_VALUE;

public int maxPathSum(TreeNode root) {
    maxGain(root);
    return maxSum;
}

private int maxGain(TreeNode node) {
    if (node == null) return 0;
    int left = Math.max(maxGain(node.left), 0);
    int right = Math.max(maxGain(node.right), 0);
    maxSum = Math.max(maxSum, node.val + left + right);
    return node.val + Math.max(left, right);
}
```

4. Serialize and Deserialize Binary Tree

Problem: Design serialization/deserialization for binary tree.

```
public String serialize(TreeNode root) {
    if (root == null) return "null";
    return root.val + "," + serialize(root.left) + "," + serialize(root.right);
}

public TreeNode deserialize(String data) {
    Queue<String> queue = new LinkedList<>(Arrays.asList(data.split(",")));
    return buildTree(queue);
}

private TreeNode buildTree(Queue<String> queue) {
    String val = queue.poll();
    if (val.equals("null")) return null;
    TreeNode node = new TreeNode(Integer.parseInt(val));
    node.left = buildTree(queue);
    node.right = buildTree(queue);
    return node;
}
```

5. Invert Binary Tree

Problem: Invert a binary tree (mirror).

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = invertTree(root.right);
    root.right = invertTree(temp);
    return root;
}
```

← Previous: Graph Algorithms | Back to Main Guide | Next: Backtracking →